

University of Pittsburgh

Introduction to Operating Systems
CS 1550

Spring 2022

Sherif Khattab

ksm73@pitt.edu

(Some slides are from **Silberschatz, Galvin and Gagne ©2013)**

Announcements

- Upcoming deadlines:
  - Homework 11: due on 4/18
  - Project 3: Late due date on 4/13
    - Late penalty waived
  - Lab 4: due on 4/15
  - Quiz 3: due on 4/15
  - Homework 12: due on 4/25
  - Lab 5: due on 5/2
  - Project 4 and Quiz 4: due on 5/2
  - Bonus Opportunities
    - Bonus Homework: due on 5/2
    - Course Post-Test: due on 5/2
    - Bonus point for all when OMET response rate >= 80%

Previous lecture …

- How do device drivers program I/O devices?
- Answer: three methods
  - polling
  - Interrupts
  - DMA

Muddiest Points (1/2)

- RAIDs
  - Difference between RAID 4 and 5
- So is the log-structured file system the same as journaling, or is that different? If it is the same, does that mean all journaling systems also track file version histories?
- Interrupt driven I/O
- polling
- Other situations like that bug/feature example? Seems interesting.
- In file block cache data structures slide, how do you track hash table collisions to make the chains?
- voltage about 0/1 on solid state disk
- Can you explain again why SSDs degrade over time?
- Single bus versus dual-bus
- What tradeoff exists having larger disk caches?
- memory mapped and separate io spaces

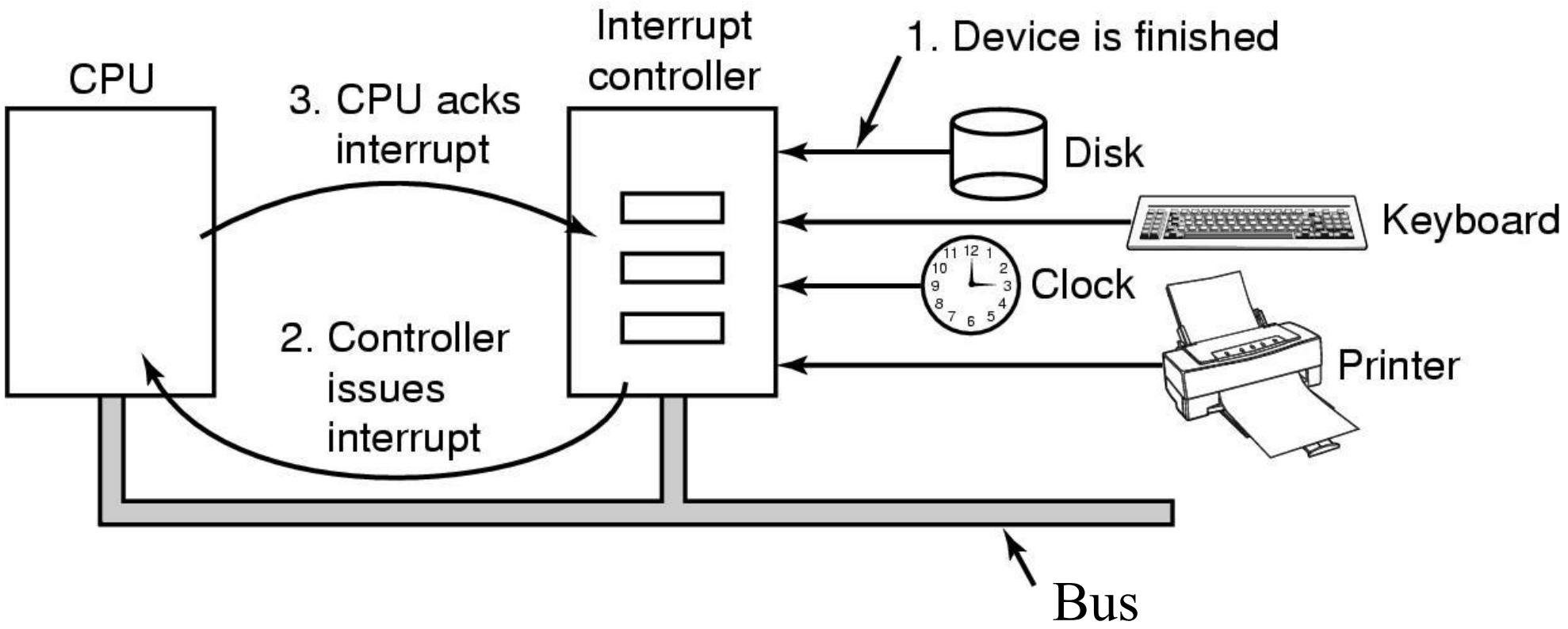
Muddiest Points (2/2)

- can you please release final preparation material a bit earlier? i'm nervous for the final and want to start studying ahead of time
- not about the lecture specifically but in general: are we allowed to work with each other on midterm exam corrections or should they be done alone?
- Are we really going to having 8am final?
- Everything was clear
- Nothing good. Need to review myself
- All good this lecture, though things went a bit fast

Polling

```
copy_from_user (buffer, p, count);  // copy into kernel buffer
for (j = 0; j < count; j++) {       // loop for each char
  while (*printer_status_reg != READY)
    ;                               // wait for printer to be ready
  *printer_data_reg = p[j];    // output a single character
}
return_to_user();
```

Hardware's view of interrupts



Interrupt-driven I/O

```
copy_from_user (buffer, p, count);
j = 0;
enable_interrupts();
while (*printer_status_reg != READY)
  ;
*printer_data_reg = p[0];
scheduler(); // and block user
```

Code run by system call

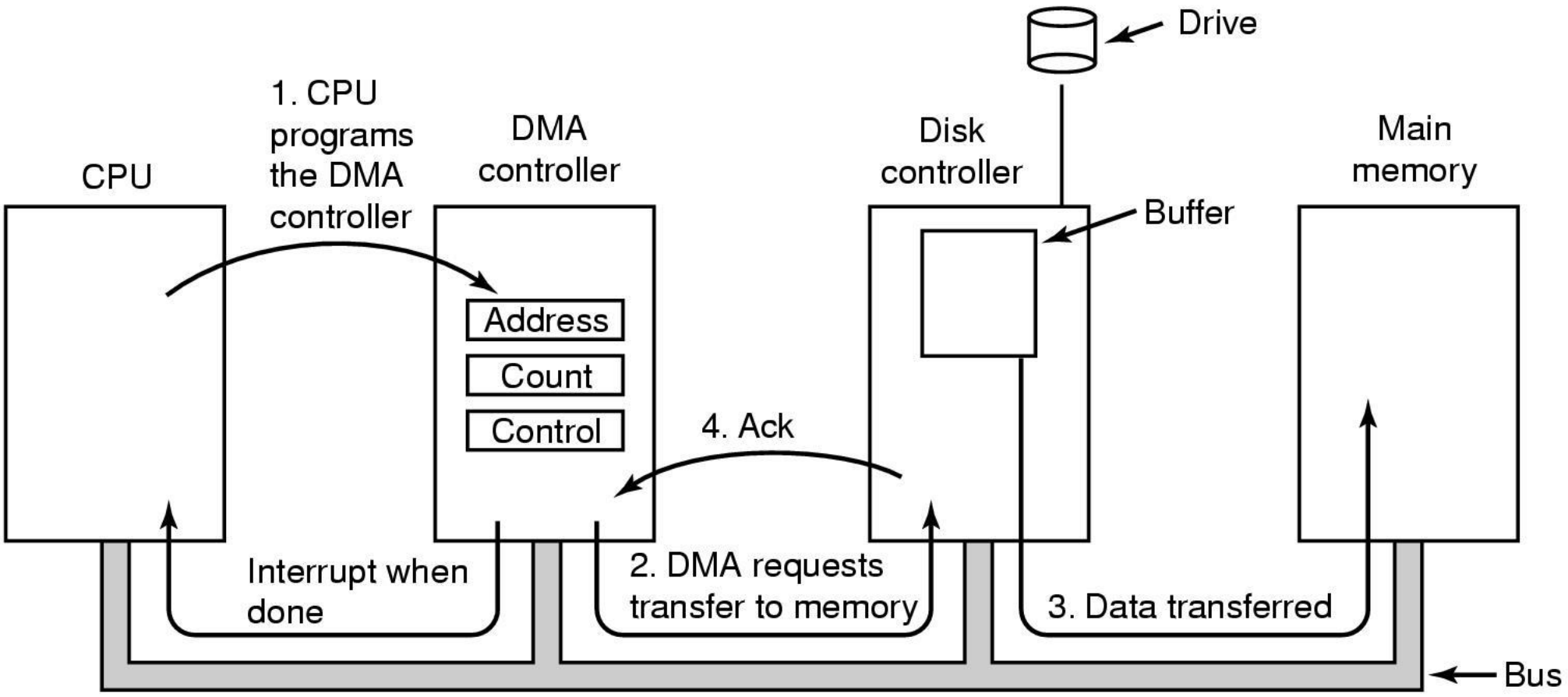
```
if (count == 0) {
  unblock_user();
} else {
  j++;
  *printer_data_reg = p[j];
  count--;
}
acknowledge_interrupt();
return_from_interrupt();
```

Code run at interrupt time
(Interrupt handler)

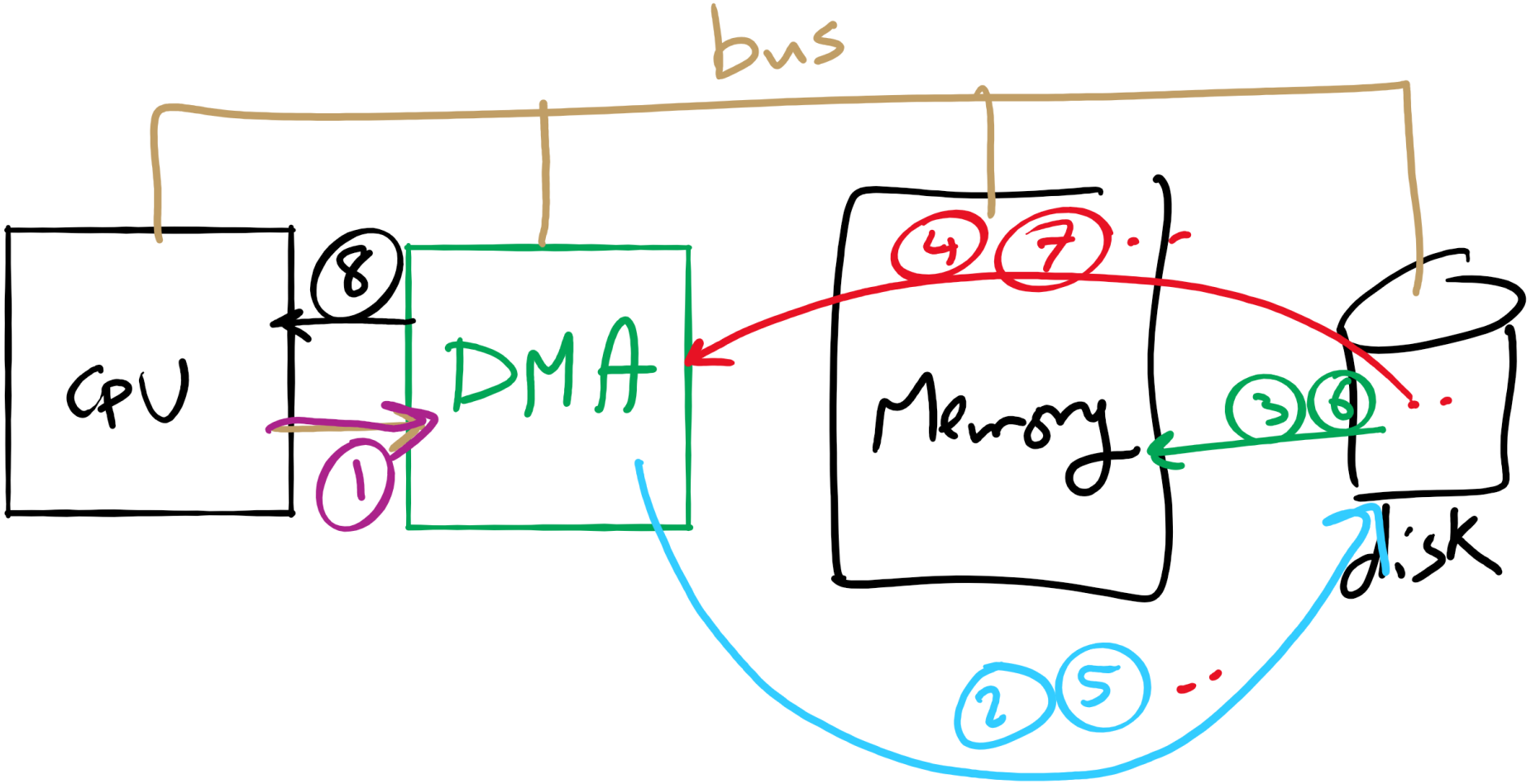
What happens on an interrupt

- Set up stack for interrupt service procedure
- Ack interrupt controller, reenable interrupts
- Copy registers from where saved
- Run service procedure
- (optional) Pick a new process to run next
- Set up MMU context for process to run next
- Load new process' registers
- Start running the new process

Direct Memory Access (DMA) operation



DMA



I/O using DMA

```
copy_from_user (buffer, p, count);
set_up_DMA_controller();
scheduler(); // and block user
```

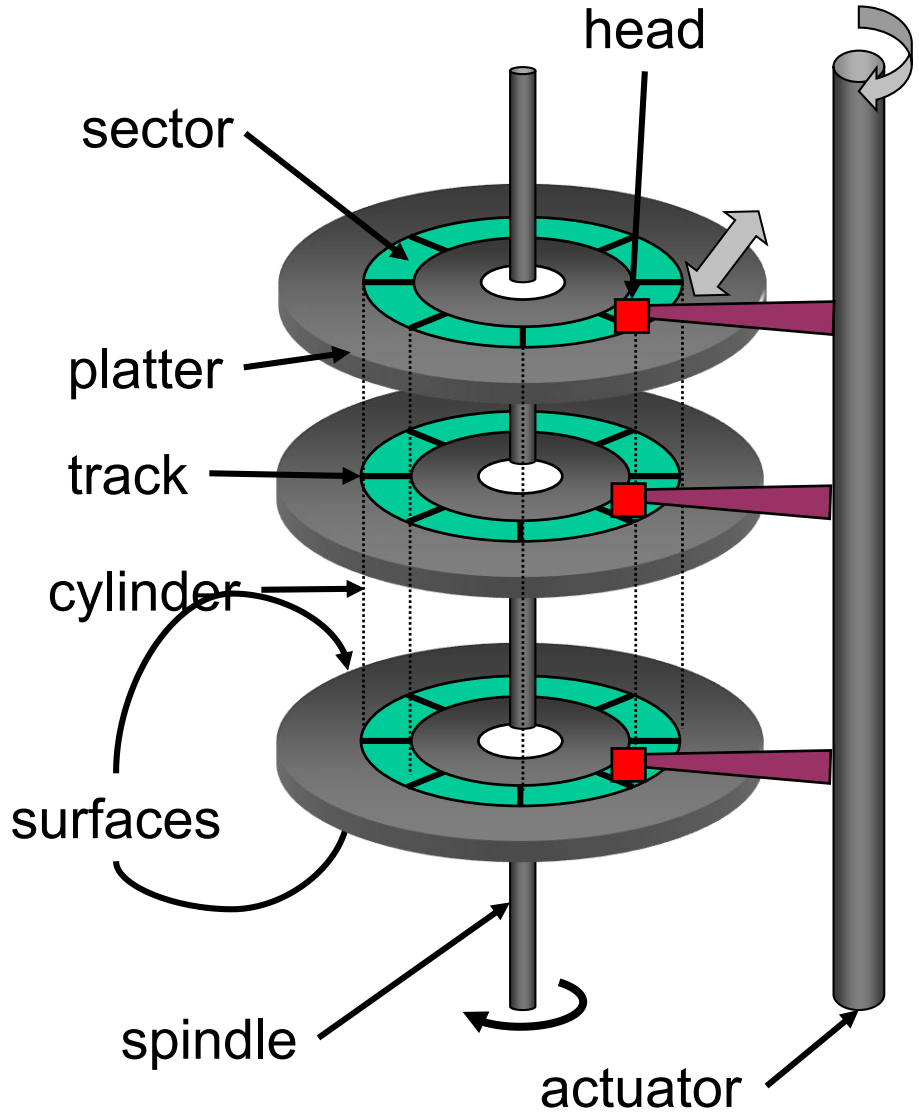
Code run by system call

```
acknowledge_interrupt();
unblock_user();
return_from_interrupt();
```

Code run at interrupt time

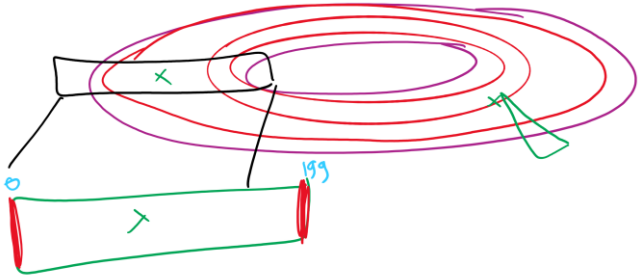
Disk drive structure

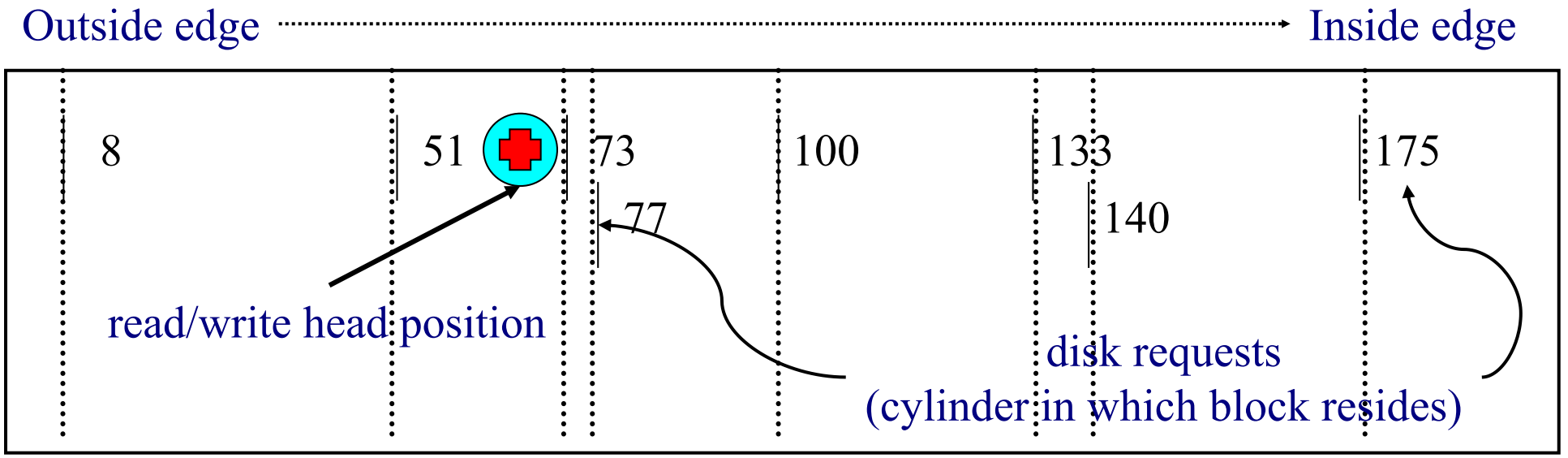
- Data stored on surfaces
  - Up to two surfaces per platter
  - One or more platters per disk
- Data in concentric tracks
  - Tracks broken into sectors
    - 256B-1KB per sector
  - Cylinder: corresponding tracks on all surfaces
- Data read and written by heads
  - Actuator moves heads
  - Heads move in unison



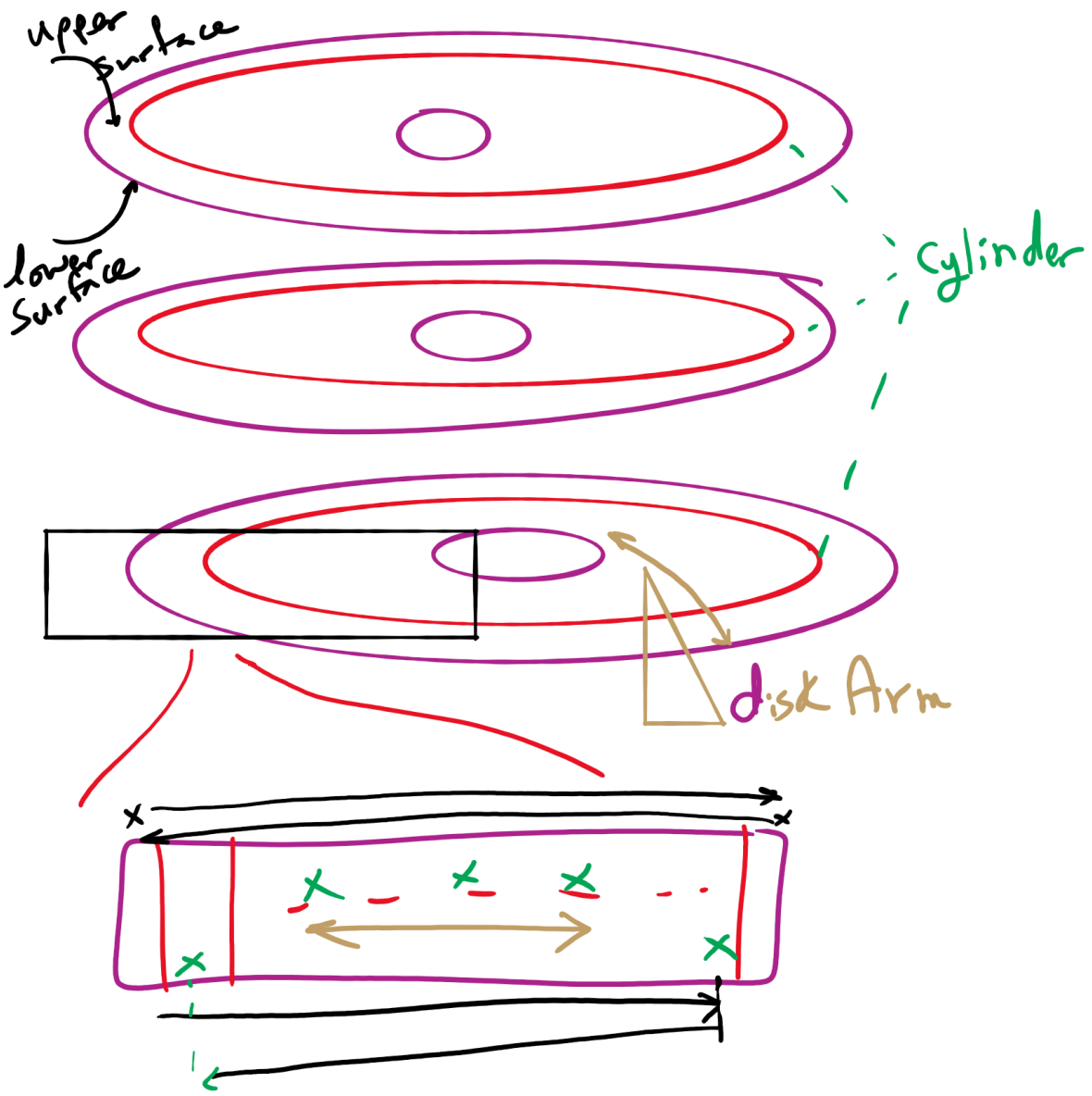
Disk scheduling algorithms

- Schedule disk requests to minimize disk seek time
  - Seek time increases as distance increases (though not linearly)
  - Minimize seek distance -> minimize seek time
- Disk seek algorithm examples assume a request queue & head position (disk has 200 cylinders)
  - Queue = 100, 175, 51, 133, 8, 140, 73, 77
  - Head position = 63



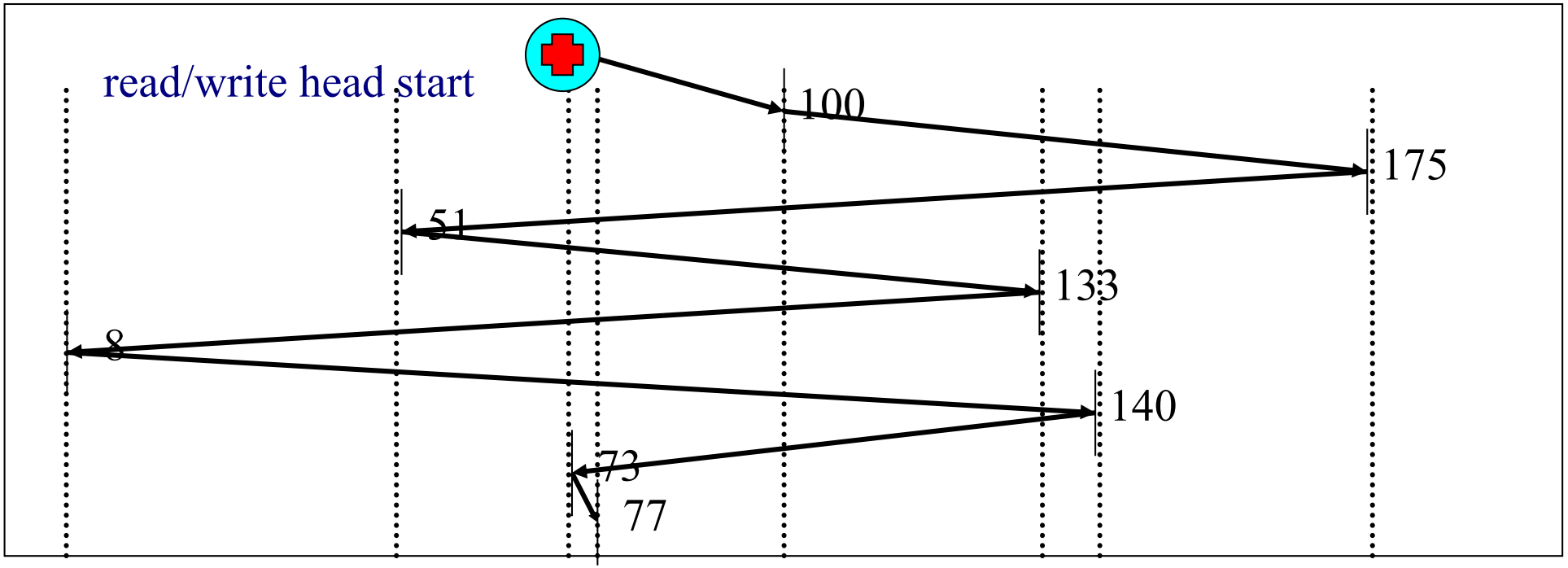


Disk Arm Scheduling

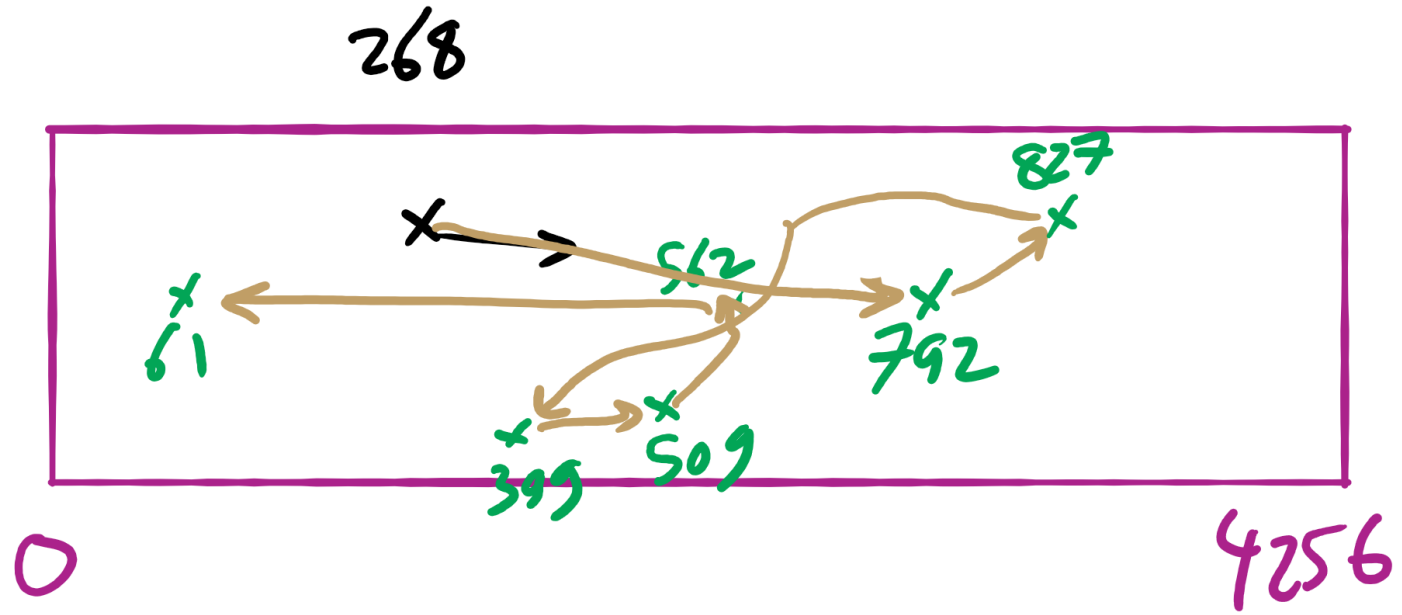


First-Come-First Served (FCFS)

- Requests serviced in the order in which they arrived
  - Easy to implement!
  - May involve lots of unnecessary seek distance
- Seek order = 100, 175, 51, 133, 8, 140, 73, 77
- Seek distance = (100-63) + (175-100) + (175-51) + (133-51) + (133-8) + (140-8) + (140-73) + (77-73) = 646 cylinders



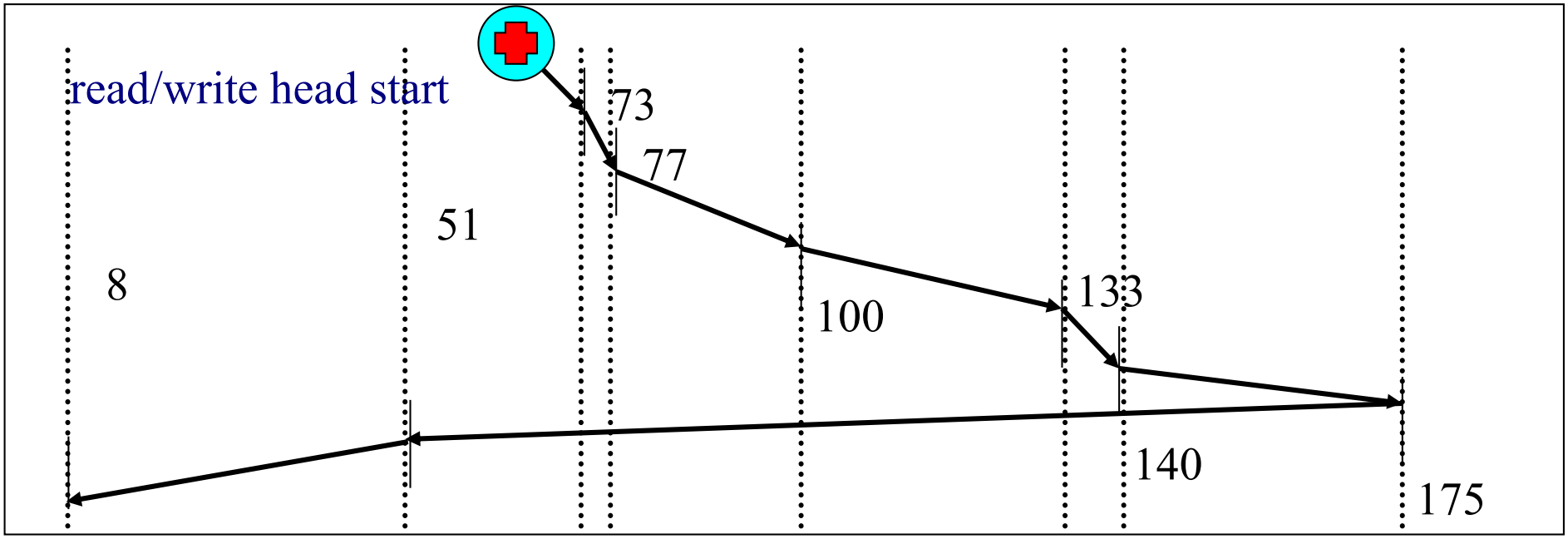
FCFS Example 2



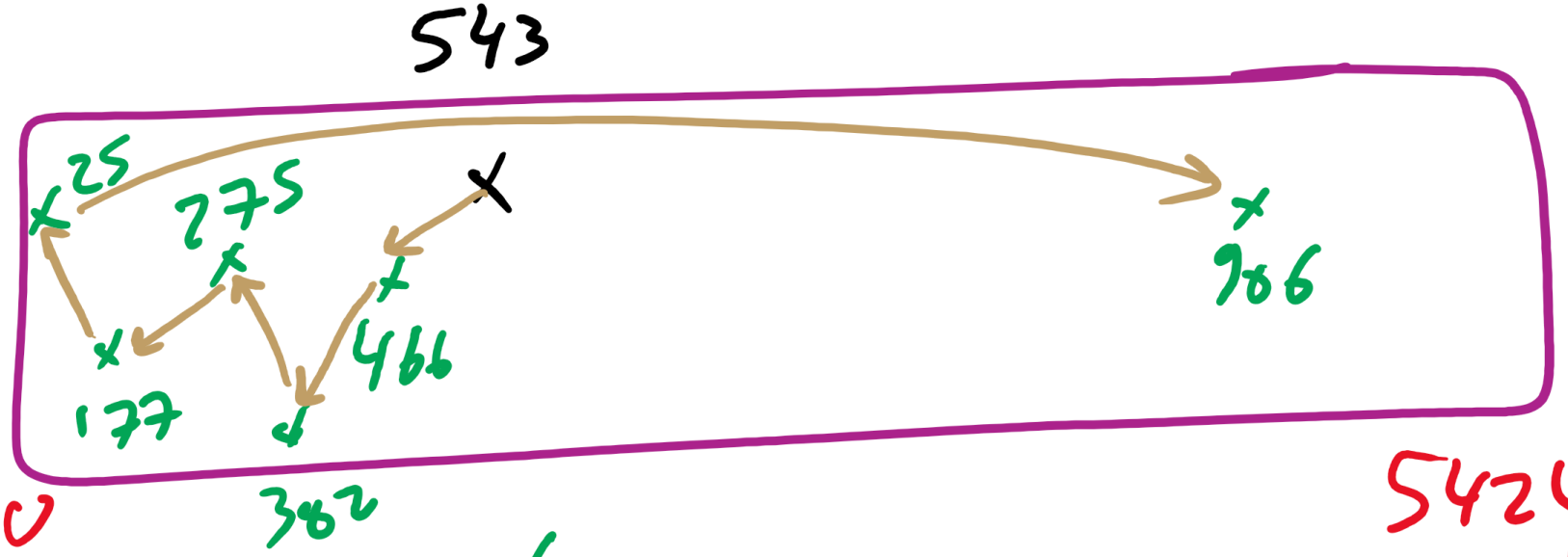
Total Seek distance = (792-268) + (827-792)
+ (827-399) + (509-399) +
(562-509) + (562-61)

Shortest Seek Time First (SSTF)

- Service the request with the shortest seek time from the current head position
  - Form of SJF scheduling
  - May starve some requests
- Seek order = 73, 77, 100, 133, 140, 175, 51, 8
- Seek distance = 10 + 4 + 23 + 33 + 7 + 35 + 124 + 43 = 279 cylinders



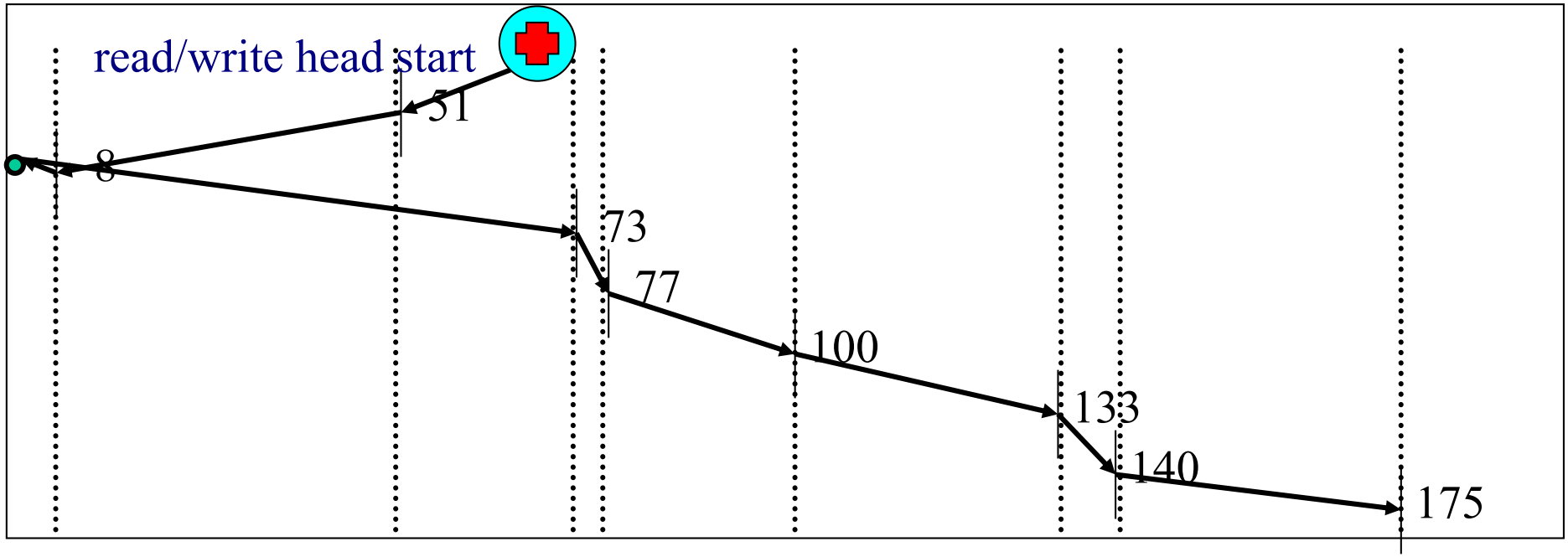
SSTF Example 2



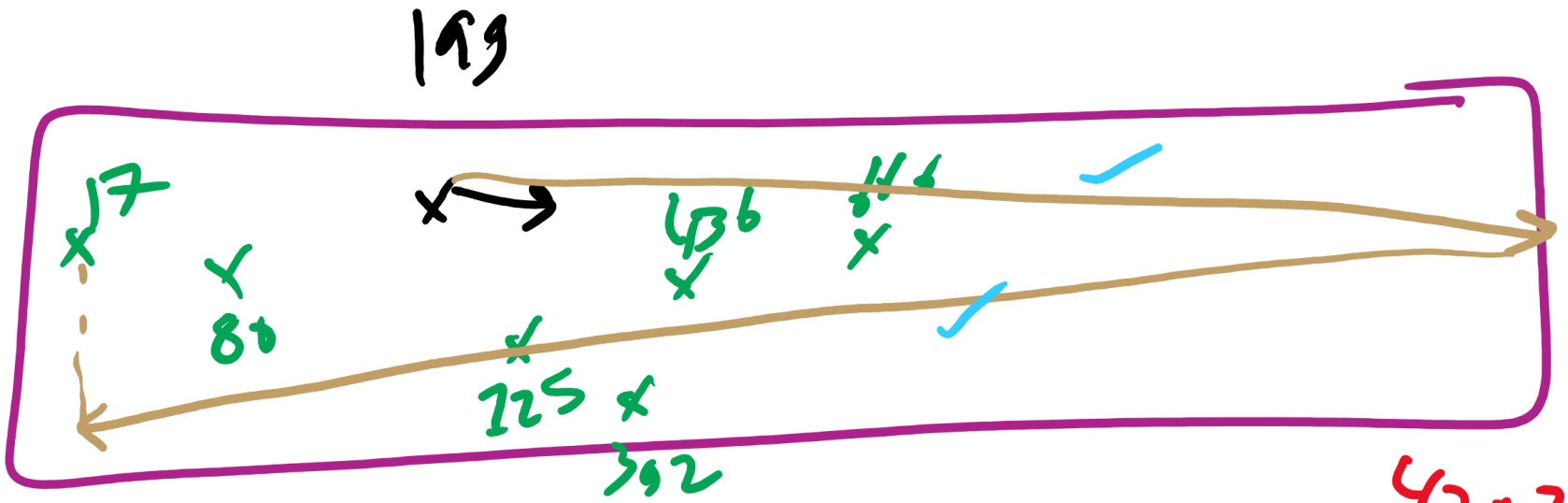
Total Seek distance = (543 − 466) + (466 − 382) + (382 − 275) + (275 − 177) + (177 − 25) + (906 − 25)

SCAN (elevator algorithm)

- Disk arm starts at one end of the disk and moves towards the other end, servicing requests as it goes
  - Reverses direction when it gets to end of the disk
  - Also known as elevator algorithm
- Seek order = 51, 8, 0 , 73, 77, 100, 133, 140, 175
- Seek distance = 12 + 43 + 8 + 73 + 4 + 23 + 33 + 7 + 35 = 238 cyls



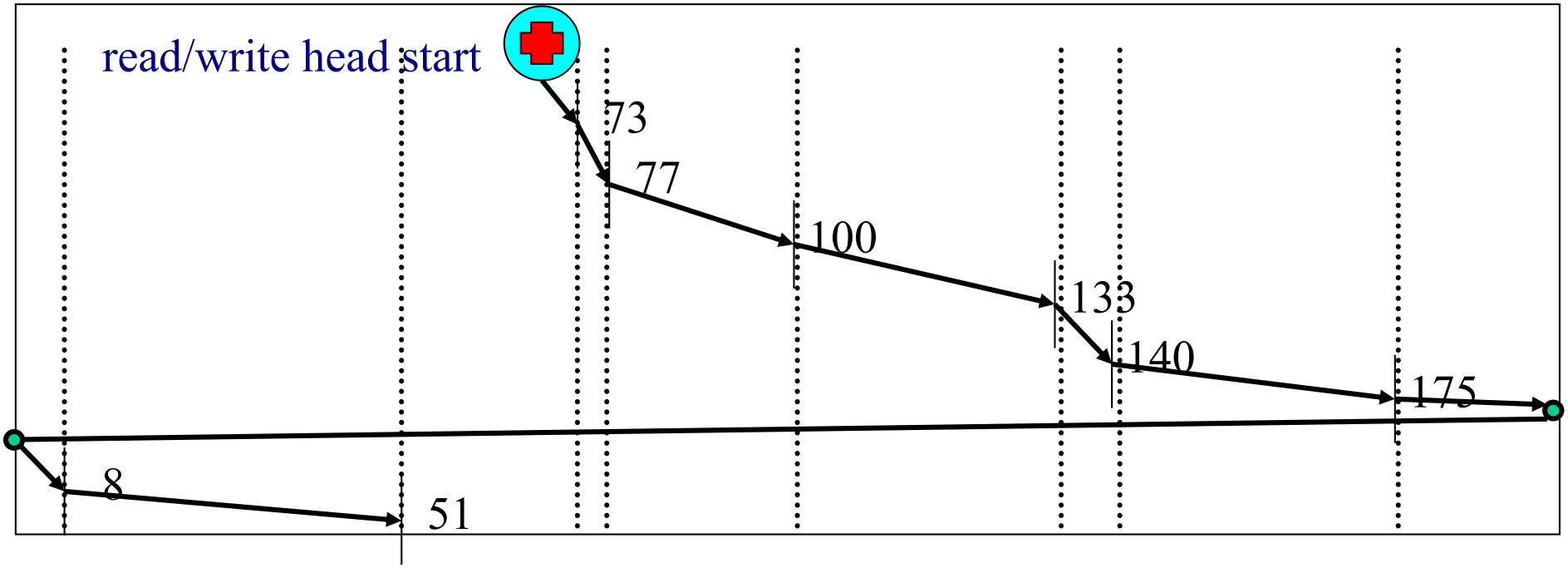
SCAN Example 2



Total Seek distance = (4303 − 199) + (4303 − 0)

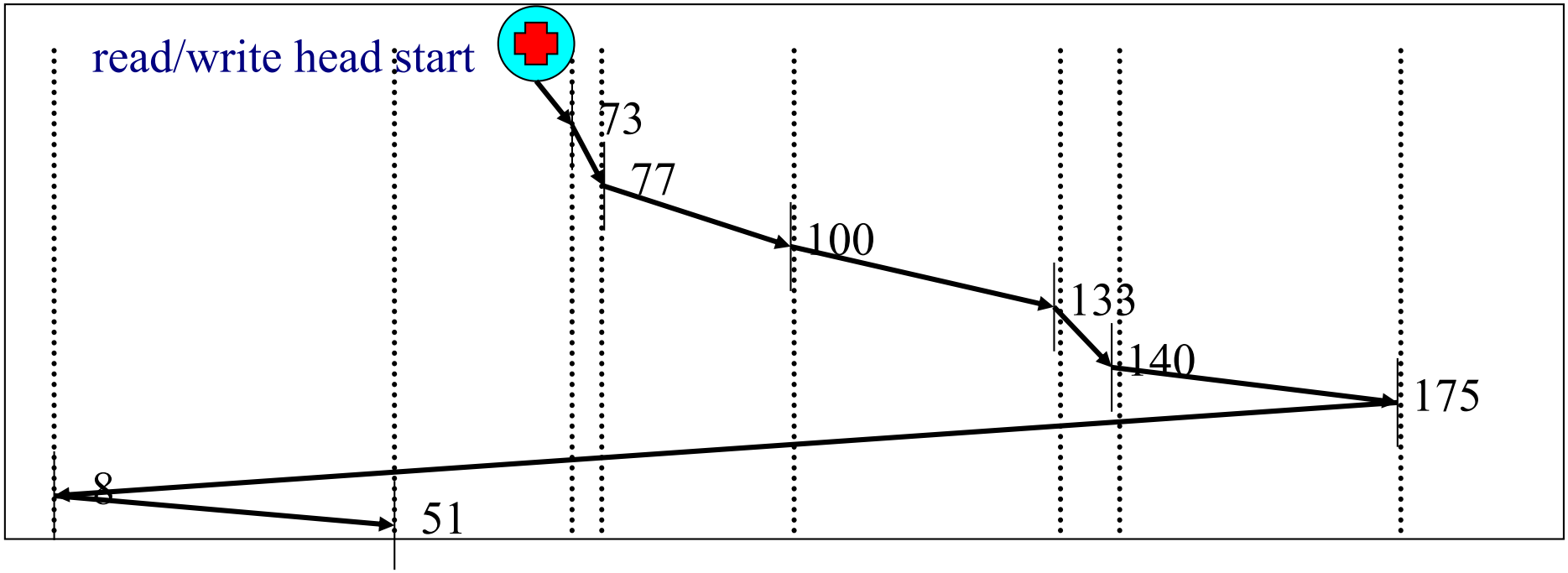
C-SCAN

- Identical to SCAN, except head returns to cylinder 0 when it reaches the end of the disk
  - Treats cylinder list as a circular list that wraps around the disk
  - Waiting time is more uniform for cylinders near the edge of the disk
- Seek order = 73, 77, 100, 133, 140, 175, 199, 0, 8, 51
- Distance = 10 + 4 + 23 + 33 + 7 + 35 + 24 + 199 + 8 + 43 = 386 cyls

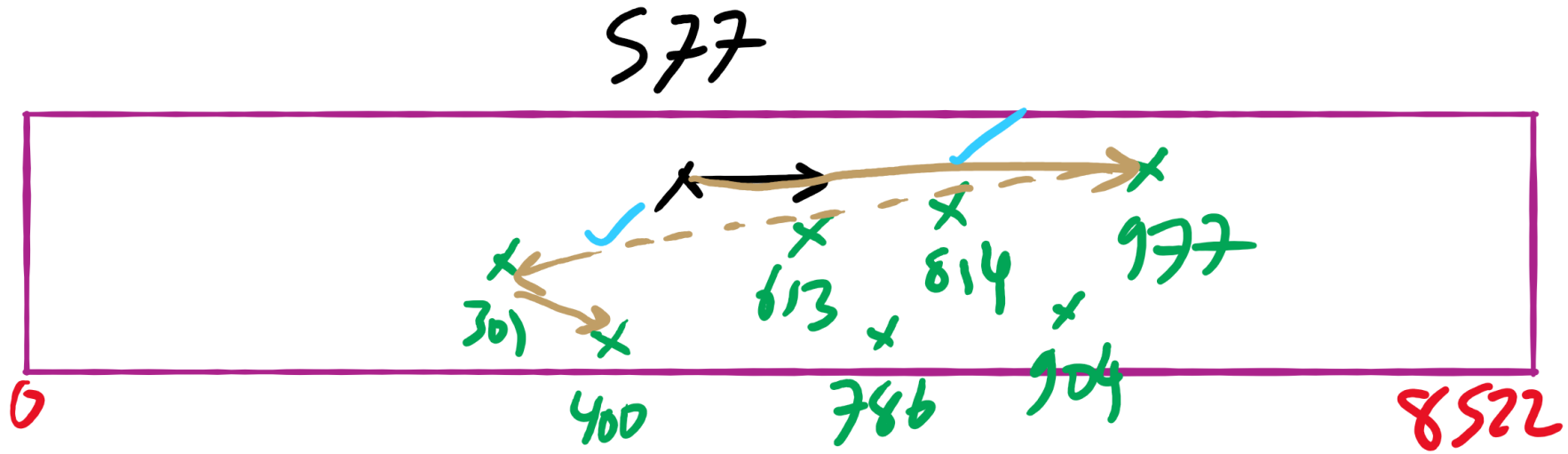


C-LOOK

- Identical to C-SCAN, except head only travels as far as the last request in each direction
  - Saves seek time from last sector to end of disk
- Seek order = 73, 77, 100, 133, 140, 175, 8, 51
- Distance = 10 + 4 + 23 + 33 + 7 + 35 + 167 + 43 = 322 cylinders



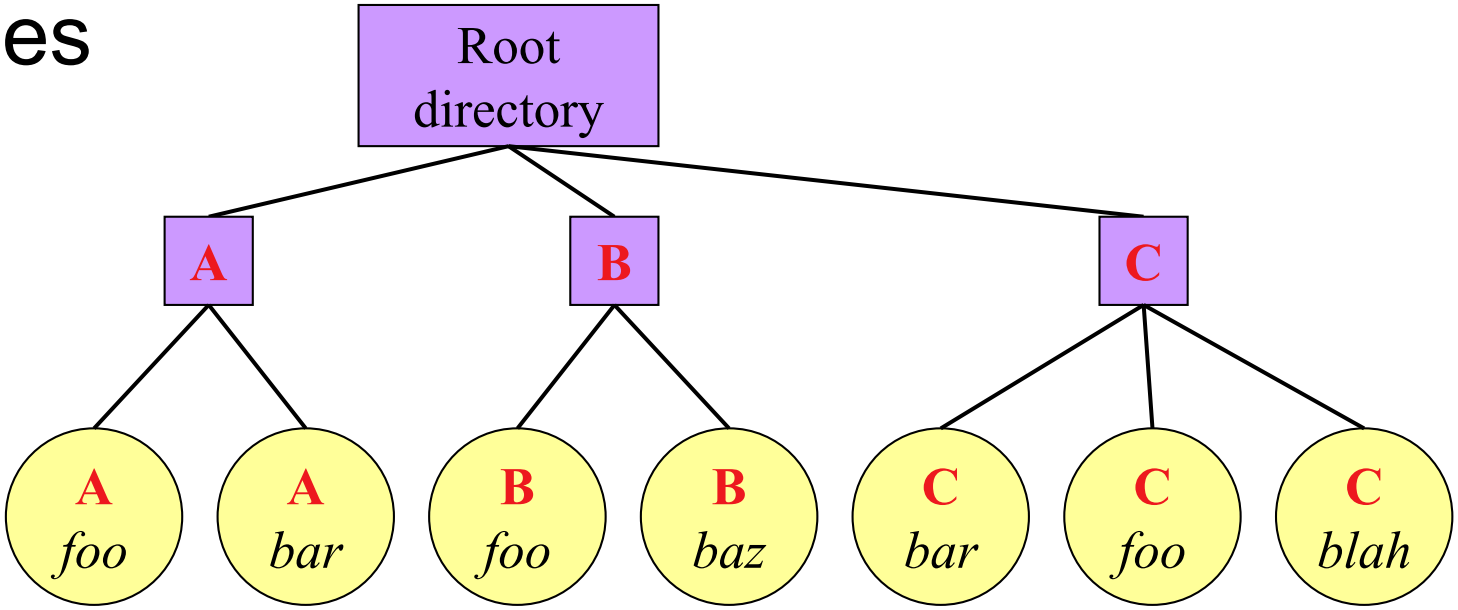
C-LOOK Example 2



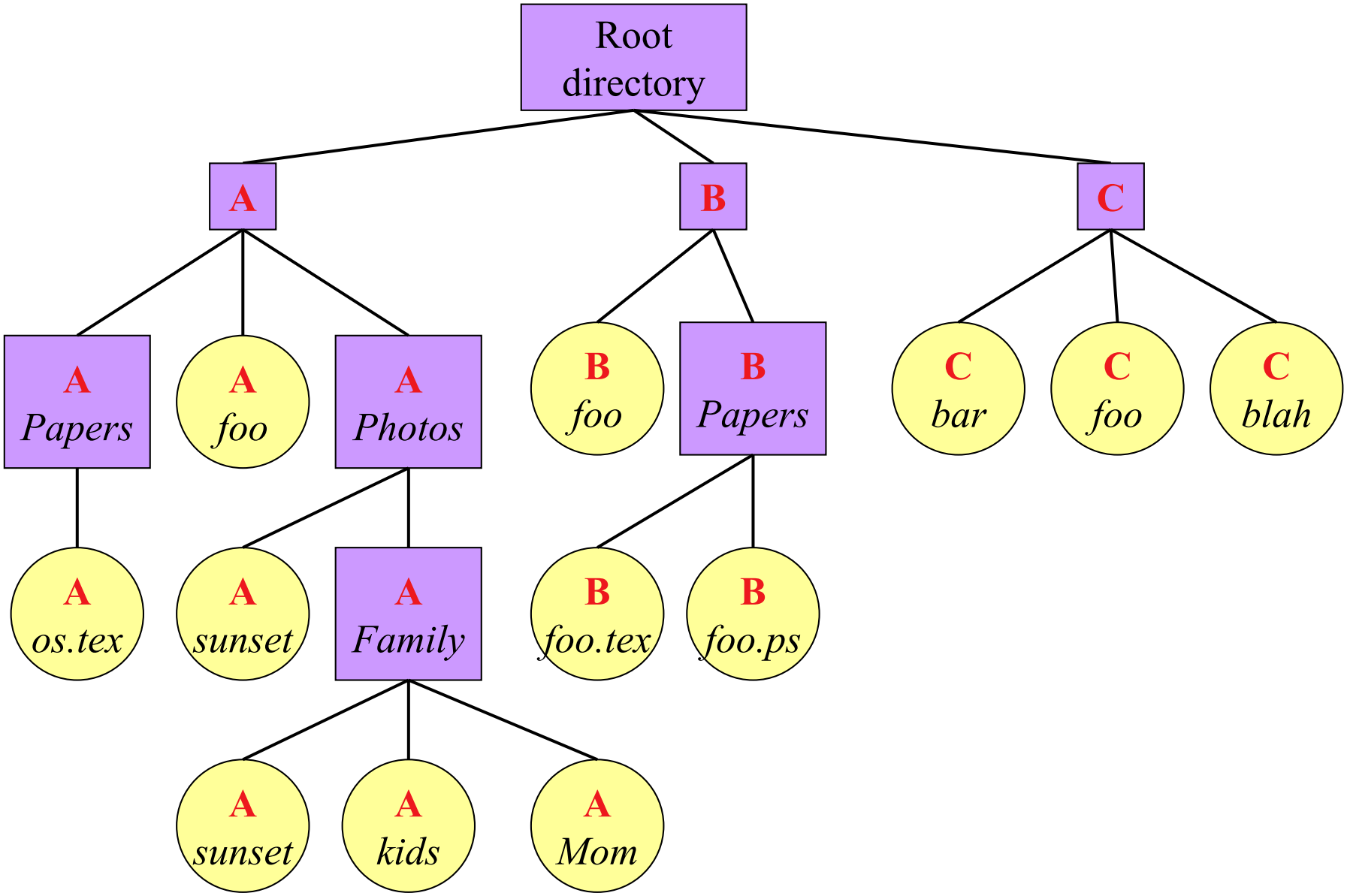
Total Seek distance = (977 − 577) +
(977 − 301) +
(400 − 301)

Two-level directory system

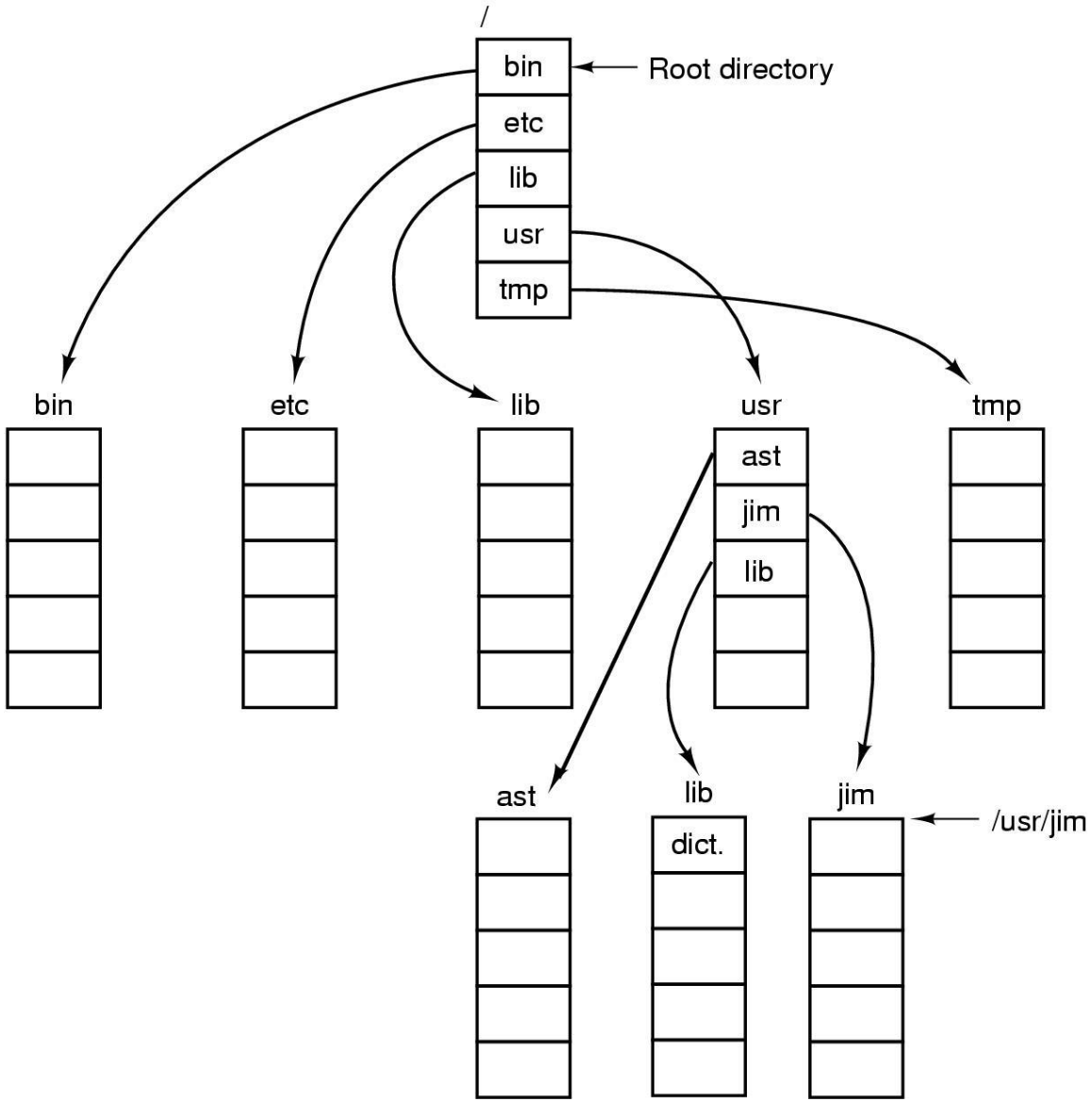
- Solves naming problem: each user has her own directory
- Multiple users can use the same file name
- By default, users access files in their own directories
- Extension: allow users to access files in others' directories



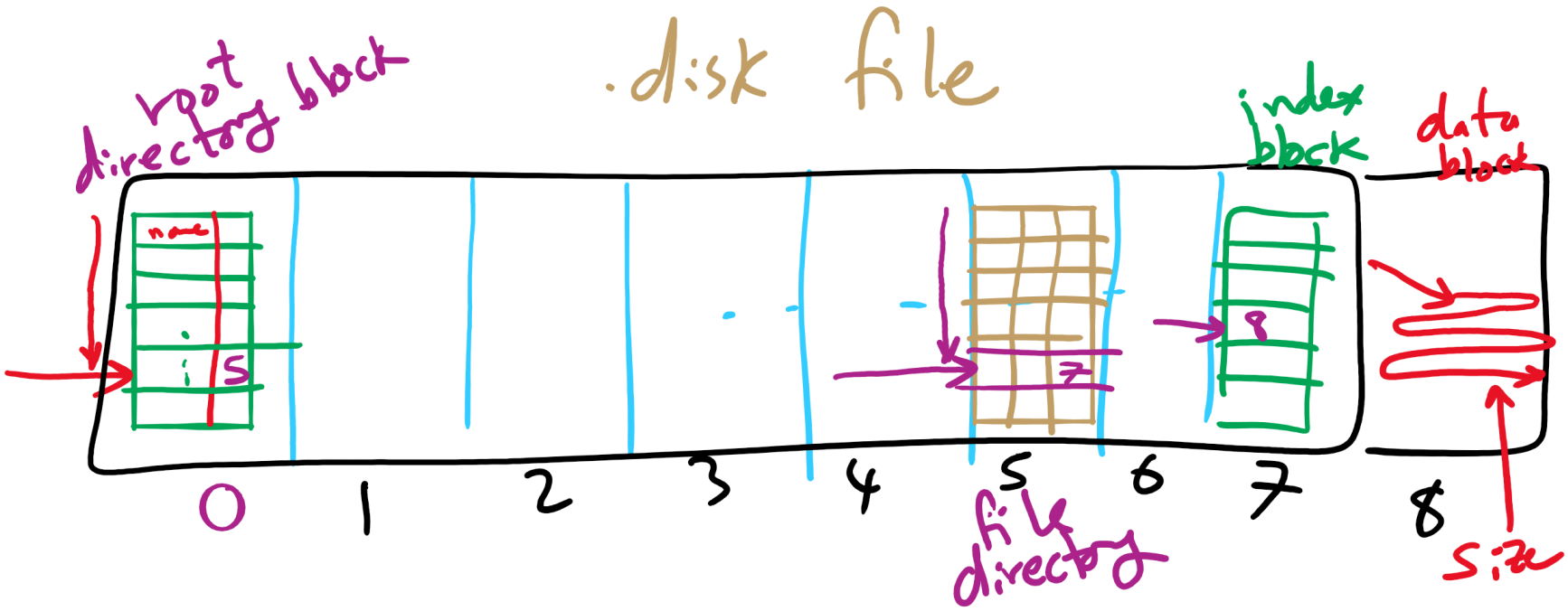
Hierarchical directory system



Unix directory tree



Project 4: Directory Traversal



Project 4 Hint

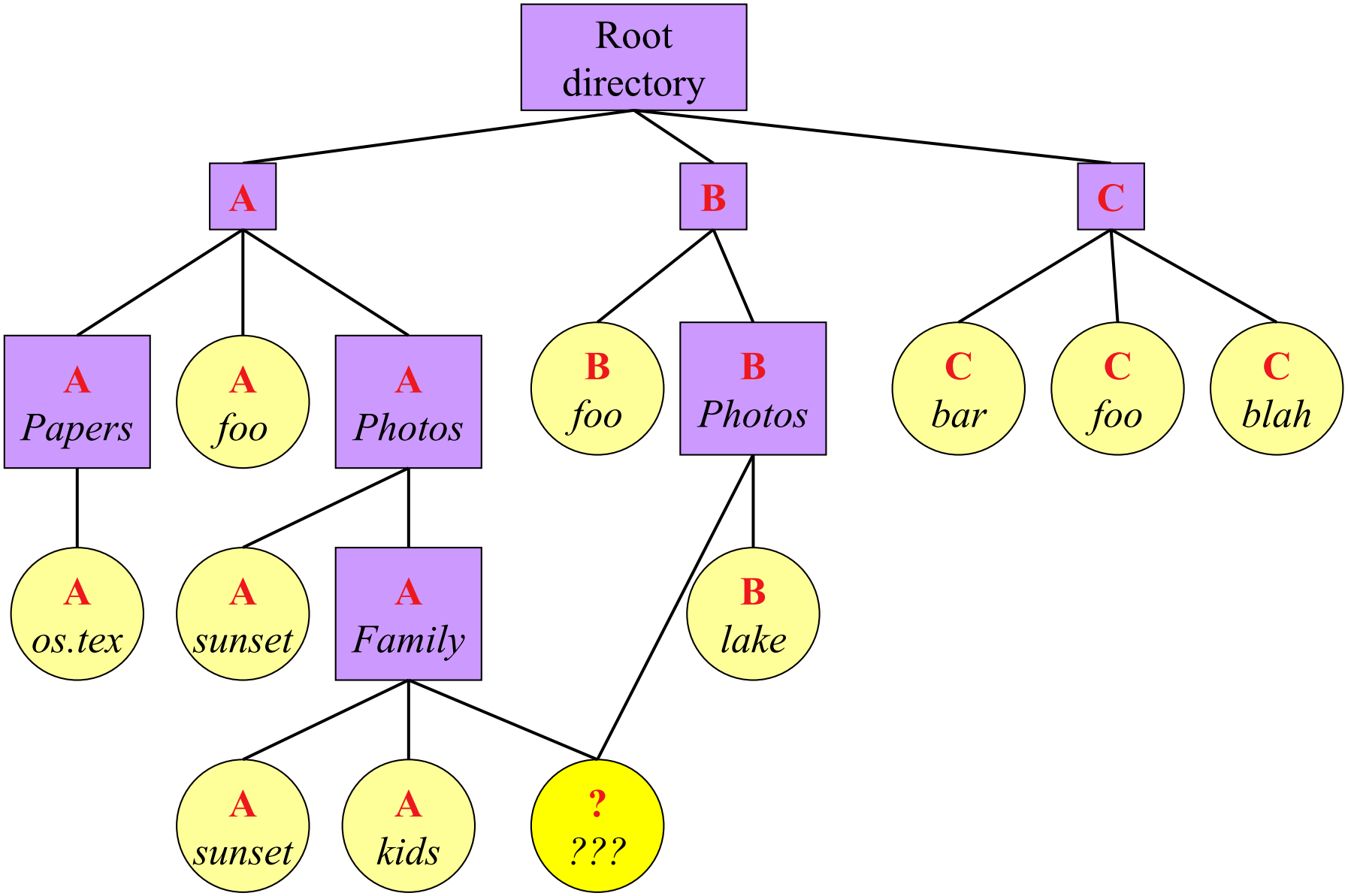








Sharing files



Solution: use links

- A creates a file, and inserts into her directory
- B shares the file by creating a link to it
- A unlinks the file
  - B still links to the file
  - Owner is still A (unless B explicitly changes it)

